

Paso 1 - El primer programa | Solucionario

Las cinco soluciones del paso 1. Tres son programas y dos son **errores de compilación provocados a propósito**: en esos, la 'salida' es lo que dice `javac`.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso01` y `javac *.java`.

B1.1 - Tu ficha

Tres `System.out.println`, uno por línea de la ficha. El único requisito escondido es el del paso 1: **el archivo se llama igual que la clase**.

▼ Ver la solución (B1_1_Ficha.java)

B1_1_Ficha.java

```
public class B1_1_Ficha {  
    public static void main(String[] args) {  
        System.out.println("Ana Perez");  
        System.out.println("Ingenieria en Informatica");  
        System.out.println("Seccion 003D");  
    }  
}
```

Salida:

```
Ana Perez  
Ingenieria en Informatica  
Seccion 003D
```

Lo que importa. Que hayas escrito las cinco líneas de estructura (la clase, el `main`, las dos llaves) sin copiarlas. Si te faltó un `;` o una llave, ese error es el ejercicio de verdad: léelo completo antes de arreglarlo.

B1.2 - Cambiarle el nombre a la clase

Al renombrar **solo la clase** y dejar el archivo como estaba, `javac` se planta. El mensaje dice exactamente qué pasa y qué espera.

Lo que dice javac:

```
$ javac B1_2_Renombrar.java
B1_2_Renombrar.java:16: error: class B1_2_RenombrarAlumno is public, should be
declared in a file named B1_2_RenombrarAlumno.java
public class B1_2_RenombrarAlumno {
      ^
1 error
```

La respuesta del punto 4: dos cosas. El nombre de la clase y el nombre del archivo, siempre juntos. En un proyecto de verdad son más, porque también hay que corregir **cada línea de otras clases que la usan** — eso lo vas a sufrir en el B1.4 y en la demo 1, donde `Demo01.java` dice `new Perro(...)`.

El `public` es lo que obliga: una clase **no** pública puede llamarse distinto que su archivo. Por eso la demo 10 puede meter tres excepciones en un solo archivo.

B1.3 - Caza de errores

Son **cuatro** defectos, pero `javac` no los muestra juntos: un error de llaves impide seguir revisando, así que aparecen en cinco rondas. Esta es la secuencia real, arreglando de a uno.

▼ Ver la solución (Practica.java)

Practica.java

```
public class Practica {
    public static void main(String[] args) {
        System.out.println("Primera linea");
        System.out.println("Segunda linea");
    }
}
```

Las cinco rondas de javac:

```

== ronda 1: como viene
Practica.java:15: error: '{' expected
public class Practica
           ^
Practica.java:16: error: class, interface, enum, or record expected
    public static void main(String[] args) {
           ^
2 errors

== ronda 2: agrego el { de la clase
Practica.java:17: error: ';' expected
        System.out.println("Primera linea")
                        ^
Practica.java:18: error: reached end of file while parsing
        system.out.println("Segunda linea");
                        ^
2 errors

== ronda 3: agrego el ; que faltaba
Practica.java:18: error: reached end of file while parsing
1 error

== ronda 4: cierro las llaves del main y de la clase
Practica.java:18: error: package system does not exist
        system.out.println("Segunda linea");
        ^
1 error

== ronda 5: System con mayuscula -> compila
Primera linea
Segunda linea

```

Lo que importa, y es la lección más útil del paso 1:

- El error de la **mayúscula de system** **no aparece hasta la ronda 4**. Mientras falten llaves, `javac` ni siquiera llega a revisar qué significa cada palabra: primero arma la estructura del archivo (eso es *parsear*) y recién después mira los nombres. Por eso un `}` olvidado esconde todos los demás errores.
- El `class, interface, enum, or record expected` de la ronda 1 **no es un cuarto defecto**: es el eco del `{` que falta.
- `reached end of file while parsing` significa siempre lo mismo: **falta cerrar una llave**. Es de los pocos mensajes que puedes arreglar sin leer nada más.

Por eso la regla del curso: **arregla el primer error, recompila, y recién ahí mira el siguiente**. Intentar arreglar los cinco de una vez rompe código que estaba bien.

B1.4 - Dos programas en la misma carpeta

Dos archivos, cada uno con su `main`. `javac *.java` compila los dos de una pasada y deja **dos** `.class`.

▼ Ver la solución (B1_4_Uno.java, B1_4_Dos.java)

B1_4_Uno.java

```
public class B1_4_Uno {  
    public static void main(String[] args) {  
        System.out.println("Soy el uno");  
    }  
}
```

B1_4_Dos.java

```
public class B1_4_Dos {  
    public static void main(String[] args) {  
        System.out.println("Soy el dos");  
    }  
}
```

Sesión completa:

```
$ javac *.java  
$ ls  
B1_4_Dos.class  B1_4_Dos.java  B1_4_Uno.class  B1_4_Uno.java  
$ java B1_4_Uno  
Soy el uno  
$ java B1_4_Dos  
Soy el dos
```

Las respuestas.

- **Dos** `.class`, uno por clase. `javac` genera uno por cada clase que compila.

- **java no elige:** tú le pasas el nombre de la clase que quieres ejecutar, y esa tiene que tener un `main`. Una carpeta puede tener veinte clases con `main`.
- **java B1_4_Uno.class falla:** `java` recibe el nombre de la **clase**, no el del archivo. Dice `Could not find or load main class B1_4_Uno.class`.
- **java b1_4_uno también falla:** Java distingue mayúsculas de minúsculas, siempre.

B1.5 - Desafío: una sola letra

Las tres mutaciones, con el mensaje completo de cada una.

Los tres mensajes:

```
--- a) Strin[] args
B1_5_Desafio.java:2: error: cannot find symbol
    public static void main(Strin[] args) {
                           ^
    symbol:   class Strin
    location: class B1_5_Desafio
1 error

--- b) voi main
B1_5_Desafio.java:2: error: cannot find symbol
    public static voi main(String[] args) {
                     ^
    symbol:   class voi
    location: class B1_5_Desafio
1 error

--- c) publi class
B1_5_Desafio.java:1: error: class, interface, enum, or record expected
publi class B1_5_Desafio {
^
1 error
```

Las respuestas.

1. Los casos **a y b**. Los dos dicen `cannot find symbol` y abajo, en la línea `symbol:`, aparece la palabra exacta que no existe: `class Strin`, `class voi`. Java supone que escribiste el nombre de **una clase que no encuentra**, y te lo dice.
2. El caso **c**. Solo dice `class, interface, enum, or record expected`: qué esperaba, no qué encontré.

3. Porque `publi` no es una palabra reservada, así que Java la lee como el **nombre de un tipo**, y entonces la línea `publi class ...` deja de parecer el comienzo de una clase. Al perder el ancla, el compilador ya no entiende el archivo completo: por eso el error apunta a la línea 1 y no menciona ninguna letra.

La moraleja para todo el curso: cuando el mensaje nombra un símbolo, el error está **ahí**. Cuando el mensaje habla de lo que *esperaba* (`expected`), el error está ahí **o más arriba**.

Volver a la [hoja del paso 1](#) o al [paso 1 de la guía](#).